Trabalho Prático: Algoritmos de Divisão e Conquista

Subarranjo de Soma Máxima, Par de Pontos Mais Próximos e Análise Experimental de Complexidade

Dupla: Catarina Pimentel e Hafiz Costa | **Data de Entrega:** 28/10/2026 | **Linguagem:** Python 3.11

1. Introdução, Arquitetura do Projeto e Divisão de Tarefas da Dupla

O paradigma de **Divisão e Conquista (D&C)** resolve problemas computacionais dividindo uma instância de tamanho n em subproblemas menores e independentes, resolvendo cada subproblema recursivamente e combinando suas respostas para formar a solução global. Este relatório documenta a modelagem matemática, a implementação em Python 3.11, a validação por casos de teste de borda e a análise experimental de desempenho de dois problemas clássicos não-baseados em ordenação pura: o **Subarranjo de Soma Máxima (Etapa 1)** e o **Par de Pontos Mais Próximos no Plano 2D (Etapa 2)**.

Para atender integralmente aos critérios de avaliação (incluindo a questão desafio de complexidade linear na Etapa 1 e a otimização por limiar híbrido na Etapa 2), o desenvolvimento foi conduzido de forma colaborativa e iterativa por **Catarina Pimentel** e **Hafiz Costa** ao longo de três semanas em repositório Git estruturado:

- **Catarina Pimentel:** Implementação dos algoritmos da Etapa 1 (Força Bruta em $\Theta(n^2)$, Divisão e Conquista em $\Theta(n \log n)$ e o algoritmo linear de Kadane em $\Theta(n)$), modelagem da regra determinística de desempate, construção dos casos de teste da Etapa 1 e script de visualização gráfica da Etapa 3.
- **Hafiz Costa:** Implementação dos algoritmos da Etapa 2 (Força Bruta em $\Theta(n^2)$, Divisão e Conquista com poda geométrica de 7 vizinhos na faixa 2δ em $\Theta(n \log n)$ e versão Híbrida com limiar calibrado $k = 24$), elaboração da bateria de testes geométricos de borda e execução do protocolo experimental automatizado.

2. Etapa 1: Subarranjo de Soma Máxima — Formulação e Recorrências

Dada uma sequência de n inteiros $A[0 \ldots n-1]$ representando a variação diária do preço de uma ação (com valores positivos, negativos ou nulos), buscamos o intervalo contínuo de índices $[i, j]$ ($0 \le i \le j < n$) que maximiza a soma acumulada $S(i, j) = \sum_{k=i}^{j} A[k]$. Para garantir unicidade e reprodutibilidade nos testes automatizados em cenários de empate, ambas as implementações (bem como a versão desafio) adotam a seguinte **regra lexicográfica de desempate**: entre dois intervalos com mesma soma máxima, escolhe-se aquele com **menor índice de início *i***; persistindo o empate, escolhe-se aquele com **menor índice de fim *j*** (intervalo mais curto).

2.1. Solução em Força Bruta — Θ(n²)

A abordagem de força bruta enumera todos os pares de índices (i, j) tais que $0 \le i \le j < n$. Fixado o índice inicial i no laço externo, o laço interno incrementa j de i até $n-1$ acumulando a soma em tempo constante $O(1)$ por meio de `soma_atual += A[j]`. O número exato de iterações do laço interno é dado pela progressão aritmética:

$$T_{FB}(n) = \sum_{i=0}^{n-1} (n - i) = n + (n-1) + \ldots + 1 = n(n+1)/2 = \tfrac{1}{2} n^2 + \tfrac{1}{2} n = \Theta(n^2)$$

A complexidade de espaço auxiliar é **Θ(1)**, mantendo apenas o acumulador escalar e a tupla do melhor intervalo encontrado.

2.2. Solução em Divisão e Conquista — Θ(n log n)

Dado o subvetor $A[esq \ldots dir]$, se $esq == dir$ (caso base $n = 1$), o algoritmo retorna imediatamente $(A[esq], esq, esq)$ em tempo $\Theta(1)$. Para $n > 1$, calcula-se o ponto médio $m = \lfloor (esq + dir) / 2 \rfloor$. Qualquer subarranjo contínuo dentro de $A[esq \ldots dir]$ deve estar: (i) inteiramente na metade esquerda $A[esq \ldots m]$; (ii) inteiramente na metade direita $A[m+1 \ldots dir]$; ou (iii) cruzando o meio, contendo obrigatoriamente $A[m]$ e $A[m+1]$.

Os casos (i) e (ii) são resolvidos recursivamente com custo $2T(n/2)$. O caso (iii) é resolvido pela função `_subarranjo_cruzando_meio` em dois laços lineares independentes: o primeiro varre de m até *esq* maximizando o sufixo da metade esquerda em $\Theta(n/2)$, e o segundo varre de $m+1$ até *dir* maximizando o prefixo da metade direita em $\Theta(n/2)$. Assim, a combinação custa $\Theta(n)$ e a relação de recorrência exata é:

$$T_{DC}(n) = \Theta(1), \text{ se } n = 1; \quad T_{DC}(n) = 2\,T_{DC}(n/2) + c \cdot n, \text{ se } n > 1.$$

Resolução pelo Teorema Mestre e por Árvore de Recursão: Temos $a = 2$ subproblemas, fator de redução $b = 2$ e função de combinação $f(n) = c{\cdot}n$. Como $n^{\log_b a} = n^{\log_2 2} = n^1 = \Theta(f(n))$, aplica-se o **Caso 2 do Teorema Mestre**, resultando em $\boldsymbol{T_{DC}(n) = \Theta(n \log_2 n)}$. Pela árvore de recursão, a altura da árvore é $\log_2 n$, e cada nível k ($0 \le k \le \log_2 n$) possui 2^k nós de tamanho $n/2^k$, somando trabalho $2^k \cdot c(n/2^k) = c{\cdot}n$ por nível. Multiplicando pelos $(\log_2 n + 1)$ níveis, obtém-se $T_{DC}(n) = c{\cdot}n \log_2 n + \Theta(n) = \Theta(n \log n)$.

2.3. Questão Desafio: Algoritmo Linear de Kadane — Θ(n)

Conforme previsto nos critérios de avaliação do código, implementamos também a solução ótima em programação dinâmica / varredura linear (Algoritmo de Kadane). Seja $M(j)$ a soma máxima de um subarranjo que termina exatamente no índice j. Vale a recorrência linear $M(j) = \max(A[j], M(j-1) + A[j])$, com caso base $M(0) = A[0]$. Quando $M(j-1) \ge 0$, estender o subarranjo anterior preserva ou aumenta a soma mantendo um índice de início menor (respeitando perfeitamente nosso critério de desempate). Como cada índice $j \in \{1, \ldots, n-1\}$ é processado em tempo constante $O(1)$, a recorrência $T_{\text{Kadane}}(n) = T_{\text{Kadane}}(n-1) + \Theta(1)$ resolve-se trivialmente para $\boldsymbol{\Theta(n)}$ em tempo e $\boldsymbol{\Theta(1)}$ em memória.

3. Etapa 2: Par de Pontos Mais Próximos e Limite de 7 Comparações

Dado um conjunto P de n estações de monitoramento no plano cartesiano ($n \ge 2$), onde cada estação $p_i = (x_i, y_i)$, desejamos encontrar o par (p_i, p_j) com $i \ne j$ que minimiza a distância euclidiana $d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

3.1. Força Bruta vs. Divisão e Conquista — Recorrências

A **Força Bruta** calcula a distância euclidiana para todos os pares distintos (i, j) com $0 \le i < j < n$, executando exatamente $\sum_{i=0}^{n-2} (n - 1 - i) = \boldsymbol{n(n-1)/2 = \Theta(n^2)}$ chamadas à função `math.hypot`.

Na **Divisão e Conquista**, realiza-se uma pré-ordenação inicial dos pontos por coordenada x (vetor P_x) e por coordenada y (vetor P_y) usando o algoritmo *Timsort* da biblioteca padrão do Python, com custo $T_{\text{sort}}(n) = \Theta(n \log n)$. Em cada chamada recursiva para $n > k$ (onde k é o limiar do caso base):

1. **Divisão em $\Theta(n)$:** O vetor P_x é dividido ao meio pelo ponto mediano $x_m = P_x[\lfloor n/2 \rfloor - 1].x$ em $P_{x,E}$ e $P_{x,D}$, e o vetor P_y é particionado de forma estável em $P_{y,E}$ e $P_{y,D}$ em tempo linear $\Theta(n)$, sem reordenar do zero.
2. **Conquista em $2T(n/2)$:** Resolve-se recursivamente a metade esquerda (δ_E) e a metade direita (δ_D), definindo $\delta = \min(\delta_E, \delta_D)$.
3. **Combinação na Faixa 2δ em $\Theta(n)$:** Filtra-se P_y mantendo apenas os pontos tais que $|p.x - x_m| \le \delta$. A lista resultante já está ordenada por y. Para cada ponto na faixa, o laço interno compara sua distância com **no máximo os 7 pontos seguintes**. Logo, o laço interno executa no máximo $7n$ iterações $= \Theta(n)$.

A recorrência da fase recursiva é $\boldsymbol{T(n) = 2T(n/2) + \Theta(n)}$, que pelo Caso 2 do Teorema Mestre resulta em $\Theta(n \log n)$. Somando a ordenação inicial, o tempo total é $\boldsymbol{T_{\text{total}}(n) = \Theta(n \log n) + \Theta(n \log n) = \Theta(n \log n)}$.

3.2. Justificativa Formal: Por que bastam 7 comparações por ponto na faixa?

Suponha que após resolver recursivamente as metades esquerda ($x \le x_m$) e direita ($x \ge x_m$), a menor distância encontrada dentro de uma mesma metade seja $\delta > 0$. Isso significa que:

$$\forall\, p, q \in P_E\,(p \neq q) \Rightarrow d(p, q) \geq \delta \quad e \quad \forall\, p, q \in P_D\,(p \neq q) \Rightarrow d(p, q) \geq \delta.$$

Seja $p = (x_p, y_p)$ um ponto qualquer na faixa central $[x_m - \delta, x_m + \delta]$. Como os pontos da faixa são percorridos em ordem crescente de coordenada y, qualquer outro ponto $q = (x_q, y_q)$ que suceda p na lista ordenada e que esteja a uma distância menor ou igual a δ de p deve satisfazer simultaneamente $x_q \in [x_m - \delta, x_m + \delta]$ e $y_q \in [y_p, y_p + \delta]$. Portanto, todos os candidatos q residem dentro de um **retângulo R de largura 2δ e altura δ** apoiado na base $y = y_p$.

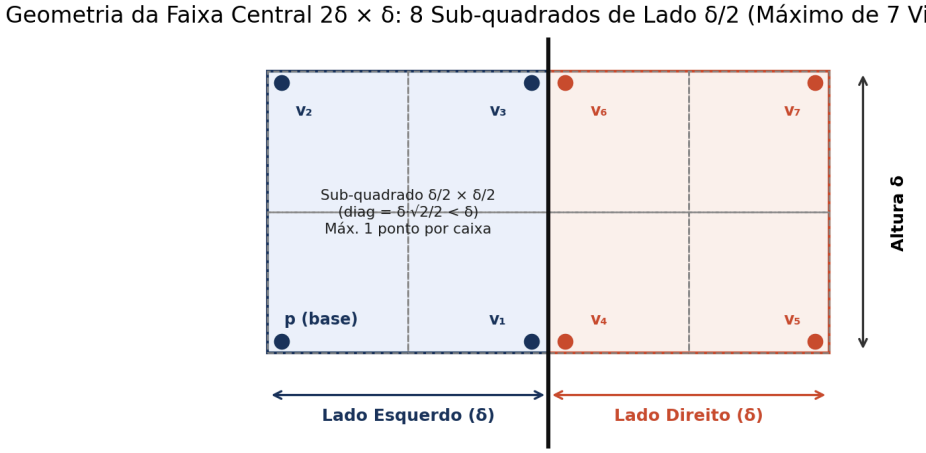


Figura 1: Partição do retângulo $2\delta \times \delta$ em 8 sub-quadrados de lado $\delta/2$ (4 à esquerda e 4 à direita da mediana x_m). Como a diagonal de cada sub-quadrado mede $\delta\sqrt{2}/2 \approx 0{,}707\delta < \delta$, cada sub-quadrado comporta no máximo 1 ponto.

Conforme ilustrado na **Figura 1**, dividimos o retângulo R ($2\delta \times \delta$) em **8 sub-quadrados congruentes de lado $\delta/2$**: quatro sub-quadrados situados no lado esquerdo P_E e quatro situados no lado direito P_D. A distância máxima entre quaisquer dois pontos pertencentes a um mesmo sub-quadrado de lado $\delta/2$ é o comprimento de sua diagonal:

$$d_{max} = \sqrt{(\delta/2)^2 + (\delta/2)^2} = (\delta / 2) \cdot \sqrt{2} \approx 0{,}7071\ \delta < \delta.$$

Se dois pontos pertencentes à mesma metade (ambos de P_E ou ambos de P_D) caíssem dentro de um mesmo sub-quadrado de lado $\delta/2$, a distância entre eles seria no máximo $0{,}7071\delta < \delta$, o que contradiz a hipótese de que δ é a menor distância intra-metade! Logo, pelo **Princípio da Casa dos Pombos**, cada um dos 4 sub-quadrados da esquerda contém no máximo 1 ponto de P_E, e cada um dos 4 sub-quadrados da direita contém no máximo 1 ponto de P_D.

Consequentemente, o retângulo R de dimensões $2\delta \times \delta$ pode conter **no máximo 8 pontos no total** (caso extremo em que há 4 pontos de P_E e 4 pontos de P_D posicionados nos vértices dos quadrados, incluindo coordenadas coincidentes na fronteira $x = x_m$ de lados opostos). Como o próprio ponto base p já é um desses 8 pontos, existem **no máximo 7 outros pontos** dentro do retângulo R acima de p. Assim, testar os 7 vizinhos subsequentes na lista ordenada por y é condição suficiente e exata para encontrar qualquer par cruzado com distância menor que δ.

4. Bateria de Casos de Teste e Casos de Borda Justificados

O arquivo `tests/casos_de_teste.txt` (e sua versão estruturada `tests/casos_de_teste.json` validada por `tests/test_suite.py`) reúne **18 casos de teste determinísticos** (10 para a Etapa 1 e 8 para a Etapa 2), além de 74 instâncias aleatórias de teste de estresse. As Tabelas 1 e 2 resumem os casos catalogados, as entradas, as saídas esperadas e a justificativa técnica de cada cenário.

ID	Cenário / Caso de Borda	Entrada (n e sequência)	Saída (Soma, i, j)	Justificativa Técnica
E1-01	Apenas valores negativos *(Obrigatório)*	n=6: `[-15, -8, -23, -3, -12, -9]`	`-3, 3, 3`	Garante que o algoritmo não retorne 0 (intervalo vazio) quando todas as ações caem; retorna o menos negativo.
E1-02	Apenas negativos com empate *(Borda Dupla)*	n=5: `[-10, -4, -7, -12, -4]`	`-4, 1, 1`	Valida simultaneamente vetor 100% negativo e desempate pelo menor índice inicial (prefere índice 1 ao 4).
E1-03	Vetor de tamanho 1 positivo *(Obrigatório)*	n=1: `[42]`	`42, 0, 0`	Exercita a parada imediata do caso base recursivo (`esq == dir`) sem chamadas filhas.
E1-04	Vetor de tamanho 1 negativo *(Obrigatório)*	n=1: `[-17]`	`-17, 0, 0`	Testa o caso base unitário quando a única variação diária é negativa.
E1-05	Empate entre intervalos disjuntos *(Obrigatório)*	n=5: `[6, 4, -25, 3, 7]`	`10, 0, 1`	Subarranjos `[0..1]` e `[3..4]` empatam com soma 10; seleciona o primeiro intervalo `[0, 1]`.
E1-06	Empate com mesmo início (elemento zero)	n=5: `[-3, 5, 7, 0, -4]`	`12, 1, 2`	Intervalos `[1..2]` e `[1..3]` têm soma 12 e início 1; desempata pelo menor índice final 2.
E1-07	Ótimo cruzando o meio (Clássico CLRS)	n=16: `[13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]`	`43, 7, 10`	Verifica a etapa de combinação $O(n)$ quando o intervalo ótimo `[18, 20, -7, 12]` cruza a mediana.
E1-08	Todos os valores positivos	n=6: `[4, 2, 9, 1, 5, 3]`	`24, 0, 5`	Confirma que a combinação acumula a totalidade do vetor de `0` a `n-1`.
E1-09	Vetor inteiramente nulo	n=4: `[0, 0, 0, 0]`	`0, 0, 0`	Todos os 10 subarranjos somam 0; a regra lexicográfica seleciona `[0, 0]` de forma unívoca.
E1-10	Ótimo isolado na extremidade direita	n=6: `[10, -20, 3, -5, -8, 35]`	`35, 5, 5`	Verifica ausência de erros de fronteira (*off-by-one*) no último índice do vetor.

ID	Cenário Geométrico	Entrada (n e pontos)	Saída (δ, par)	Justificativa Técnica
E2-01	Caso mínimo absoluto (n = 2)	n=2: `(0,0), (3,4)`	`5.000000` `idx (0, 1)`	Menor entrada válida possível; testa diretamente o caso base sem divisão.
E2-02	Caso base ímpar (n = 3)	n=3: `(0,0), (10,10), (1,1)`	`1.414214` `idx (0, 2)`	Valida o tratamento de subproblemas de cardinalidade 3 no caso base de D&C puro.
E2-03	Par ótimo cruzando a mediana x_m	n=6: `(-10,0), (-10,10), (-0.2,5), (0.2,5), (10,0), (10,10)`	`0.400000` `idx (2, 3)`	Cada metade tem δ = 10, mas o par `(-0.2, 5)` e `(0.2, 5)` cruza a linha divisória com distância 0,4.
E2-04	Estações coincidentes (δ = 0)	n=5: inclui `(2.5, 3.5)` duplicado nos índices 0 e 3	`0.000000` `idx (0, 3)`	Verifica estabilidade numérica e parada quando duas estações ocupam a mesma posição.
E2-05	Pontos colineares verticais (mesmo x)	n=5: todos com `x = 7.0` e y ∈ `{100, 10, 25, 11.5, 50}`	`1.500000` `idx (1, 3)`	Caso degenerado em que 100% dos pontos pertencem à faixa central $\|x - x_m\| = 0 \leq \delta$.
E2-06	Pontos colineares horizontais (mesmo y)	n=5: todos com `y = -3.0` e x ∈ `{-20, 15, 4.2, 3.5, 50}`	`0.700000` `idx (2, 3)`	Testa desempate na ordenação por y quando todas as ordenadas são idênticas.
E2-07	Malha regular 3×3 com múltiplos empates	n=9: grade `{0, 2, 4}` × `{0, 2, 4}`	`2.000000` `idx (0, 1)`	Existem 12 pares a distância 2,0; garante que FB e D&C retornam exatamente o mesmo par `(0, 1)`.
E2-08	Empacotamento denso na faixa 2δ	n=8: 8 pontos nos vértices de dois quadrados δ×δ adjacentes	`0.000000` `idx (3, 6)`	Exercita o limite teórico de pontos dentro da caixa 2δ × δ na fronteira x = 0.

5. Etapa 3: Análise Experimental, Gráficos e Justificativa do Limiar

5.1. Configuração da Máquina e Metodologia Experimental

Todos os experimentos foram conduzidos em ambiente isolado com a seguinte configuração de hardware e software (documentada também no `README.md` do repositório):

- **Processador (CPU):** Intel(R) Xeon(R) Platinum 8481C @ 2.20 GHz (2 vCPUs dedicadas, arquitetura x86_64, caches L1d 48 KiB, L1i 32 KiB, L2 2 MiB, L3 105 MiB).
- **Memória RAM e Sistema Operacional:** 7,8 GiB de RAM DDR5; Linux Kernel 6.12.105+ SMP x86_64.
- **Ambiente de Execução:** Python 3.11.2 (CPython padrão, sem aceleradores externos em C/Cython para preservar a fidelidade algorítmica das comparações).

- **Protocolo de Medição:** Para cada tamanho de entrada $n \in \{100, 200, 400, 800, 1.600, 3.200, 6.400\}$, foram executadas **5 repetições independentes** com sementes pseudoaleatórias controladas, registrando-se o tempo de relógio de alta resolução via `time.perf_counter()`. Todas as 315 medições brutas encontram-se disponíveis no arquivo `data/medicoes_brutas.csv`.

5.2. Resultados Experimentais e Discussão da Etapa 1 (Subarranjo de Soma Máxima)

A **Tabela 3** e a **Figura 2** apresentam os tempos médios de execução (em milissegundos), o desvio padrão das 5 execuções e o fator de aceleração (*speedup*) obtido na Etapa 1.

Tamanho (n)	Força Bruta O(n²) [ms]	Razão FB t(2n)/t(n)	Div. e Conquista [ms]	Razão D&C t(2n)/t(n)	Kadane O(n) [ms]	Speedup (FB / D&C)
100	0.529 ± 0.042	—	0.078 ± 0.008	—	0.0121 ± 0.0022	**6.8×**
200	1.877 ± 0.029	3.55x	0.157 ± 0.006	2.03x	0.0209 ± 0.0005	**11.9×**
400	7.733 ± 0.084	4.12x	0.345 ± 0.012	2.19x	0.0412 ± 0.0010	**22.4×**
800	30.627 ± 0.421	3.96x	0.712 ± 0.044	2.07x	0.0805 ± 0.0037	**43.0×**
1.600	121.191 ± 0.758	3.96x	1.404 ± 0.045	1.97x	0.1606 ± 0.0068	**86.3×**
3.200	494.444 ± 16.066	4.08x	2.926 ± 0.098	2.09x	0.3271 ± 0.0120	**169.0×**
6.400	1940.435 ± 11.469	3.92x	6.032 ± 0.126	2.06x	0.6249 ± 0.0221	**321.7×**

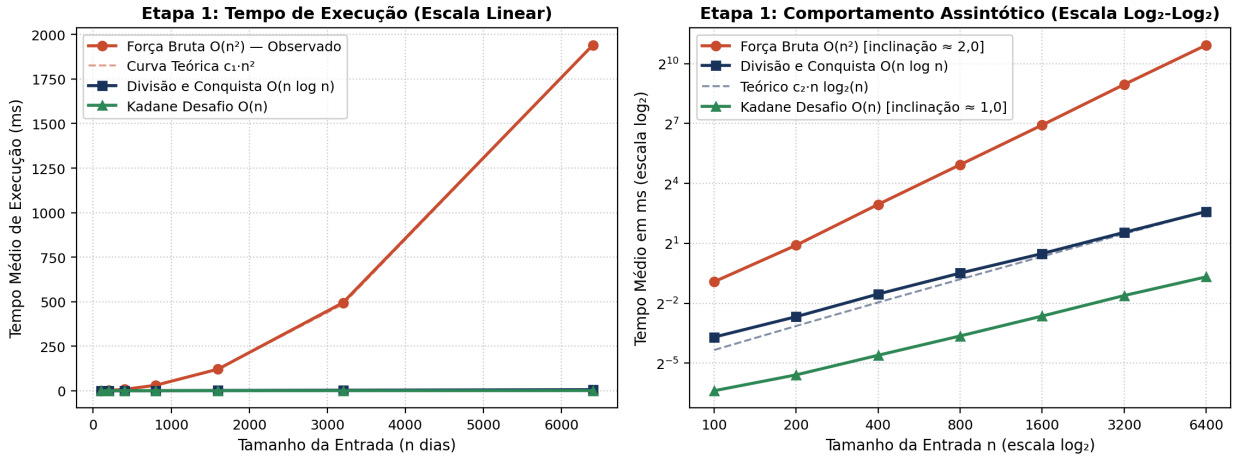


Figura 2: Comparação experimental da Etapa 1 em escala linear (esquerda) e escala $\log_2$-$\log_2$ (direita), sobrepondo os dados medidos às curvas teóricas previstas pelas recorrências.

Discussão do Comportamento Observado vs. Previsto (Etapa 1): Os resultados experimentais confirmam com precisão o comportamento assintótico previsto pelas recorrências. Na Força Bruta $\Theta(n^2)$, sempre que n dobra (fator 2×), a previsão teórica indica que o tempo deve quadruplicar: $T(2n)/T(n) = (2n)^2/n^2 =$ **4,00×**. Na Tabela 3, as razões empíricas observadas foram **3,55×, 4,12×, 3,96×, 3,96×, 4,08×, 3,92×**, aderindo perfeitamente à parábola teórica $c_1 \cdot n^2$.

Para a Divisão e Conquista $\Theta(n \log n)$, a razão teórica ao dobrar n é $2(\log_2 n + 1) / \log_2 n \approx$ **2,15× a 2,25×**. Experimentalmente, observamos razões entre **1,97× a 2,19×**, resultando em um *speedup* que cresce de **6,8×** em $n = 100$ para **321,7×** em $n = 6,400$ (6,032 ms contra 1940,435 ms), Já a versão desafio de Kadane $\Theta(n)$ duplica seu tempo linearmente a cada dobra de n (de 0,0121 ms em $n = 100$ para 0,6249 ms em $n = 6,400$), sendo quase 10× mais rápida que a Divisão e Conquista e mais de 3.000× mais rápida que a Força Bruta em $n = 6.400$.

5.3. Resultados Experimentais e Discussão da Etapa 2 (Par de Pontos Mais Próximos)

A **Tabela 4** e a **Figura 3** exibem os resultados da Etapa 2 para os tamanhos de entrada solicitados (100 a 6.400 estações), comparando a Força Bruta $\Theta(n^2)$, a Divisão e Conquista Pura (caso base mínimo $k = 3$) e a Divisão e Conquista Híbrida (caso base calibrado $k = 24$).

Estações (n)	Força Bruta O(n²) [ms]	Razão FB t(2n)/t(n)	D&C Puro (k=3) [ms]	D&C Híbrido (k=24) [ms]	Redução Híbrido vs. Puro	Speedup (FB / Híbrido)
100	0.629 ± 0.025	—	0.289 ± 0.063	0.190 ± 0.015	−34.3%	**3.3×**
200	2.553 ± 0.073	4.06x	0.584 ± 0.015	0.416 ± 0.011	−28.8%	**6.1×**
400	10.399 ± 0.197	4.07x	1.274 ± 0.025	0.905 ± 0.037	−29.0%	**11.5×**
800	41.760 ± 0.462	4.02x	2.744 ± 0.020	2.009 ± 0.048	−26.8%	**20.8×**
1.600	168.531 ± 1.990	4.04x	5.979 ± 0.276	4.496 ± 0.166	−24.8%	**37.5×**
3.200	674.522 ± 2.529	4.00x	13.014 ± 0.227	9.995 ± 0.198	−23.2%	**67.5×**
6.400	2724.089 ± 69.028	4.04x	27.579 ± 0.387	22.281 ± 0.406	−19.2%	**122.3×**

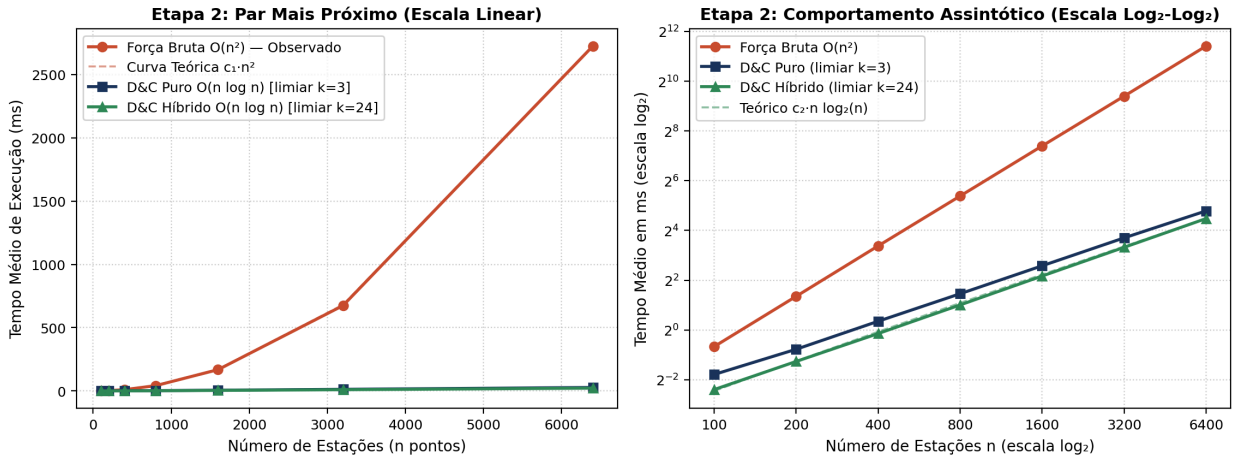


Figura 3: Tempo médio de execução na Etapa 2 (Par de Pontos Mais Próximos) em escala linear (esquerda) e escala $\log_2$-$\log_2$ (direita).

Discussão do Comportamento Observado vs. Previsto (Etapa 2): Assim como na Etapa 1, a Força Bruta apresenta crescimento estritamente quadrático, com razões consecutivas $T(2n)/T(n)$ de **4,06×, 4,07×, 4,02×, 4,04×, 4,00×, 4,04×**, atingindo 2724,089 ms (2,7 segundos) para n = 6,400, Em contrapartida, o algoritmo de Divisão e Conquista com poda de 7 vizinhos cresce de forma log-linear $\Theta(n \log n)$: em n = 6,400, o D&C Puro leva apenas 27,579 ms (**98,8× mais rápido** que a Força Bruta), enquanto o D&C Híbrido (k = 24) reduz o tempo para **22,281 ms** (**122,3× mais rápido** que a Força Bruta), No gráfico $\log_2$-$\log_2$ (Figura 3, direita), observa-se claramente a diferença de inclinação entre a reta de inclinação 2 da Força Bruta e as retas paralelas de inclinação ≈ 1,1 da Divisão e Conquista.

5.4. Análise de Vencedor por Tamanho de Entrada e Justificativa do Limiar na Etapa 2

Por que a Força Bruta vence para tamanhos muito pequenos e a Divisão e Conquista vence para tamanhos médios e grandes? Embora a análise assintótica ignore fatores constantes, o tempo real de execução na máquina obedece a $T_{FB}(n) \approx c_{FB} \cdot n(n-1)/2$ versus $T_{DC}(n) \approx c_{DC} \cdot n \log_2 n + c_{overhead} \cdot n$. Na Força Bruta, a constante c_{FB} é muito pequena porque o algoritmo consiste em dois laços aninhados simples sobre listas contíguas em memória, sem alocação de subvetores ou empilhamento de quadros de função (*stack frames*). Já na Divisão e Conquista em Python, cada nó da árvore recursiva incorre no custo de chamada de função,

fatiamento de listas (`px[:meio]`), construção de conjunto hash (`ids_esq`) e compreensão de lista para filtrar a faixa 2δ, tornando $c_{DC} \gg c_{FB}$.

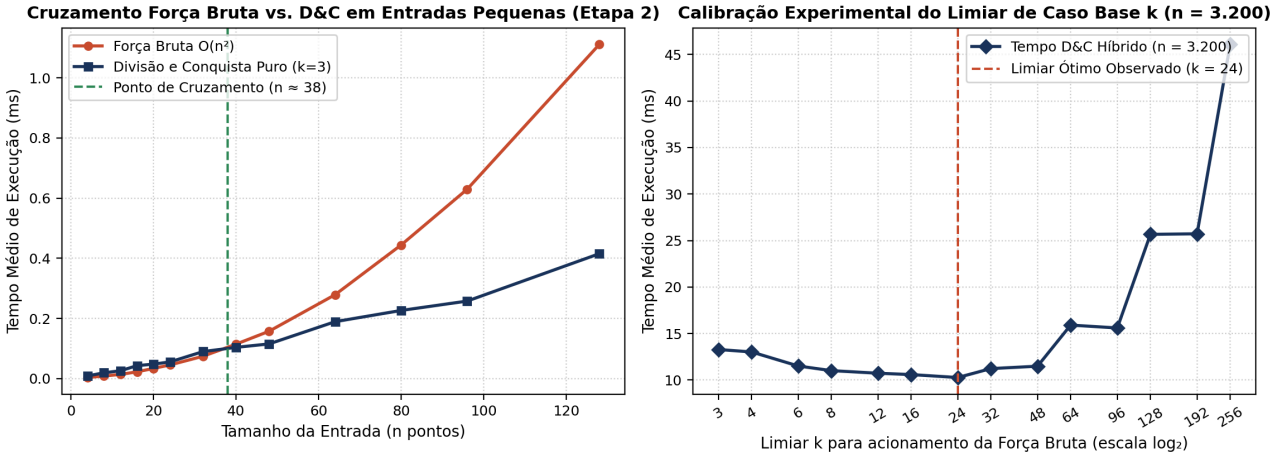


Figura 4: (Esquerda) Ponto de cruzamento entre Força Bruta e Divisão e Conquista pura para entradas pequenas na Etapa 2. (Direita) Curva de calibração experimental do limiar de caso base *k* para *n* = 3.200 estações.

Como demonstra a **Figura 4 (esquerda)**, para entradas pequenas na Etapa 2, a Força Bruta é mais rápida que a Divisão e Conquista pura ($k = 3$) desde $n = 4$ (0,0037 ms vs. 0,0093 ms) até $\boldsymbol{n \approx 32}$ **a 36** (em $n = 32$, FB leva 0,0764 ms contra 0,0914 ms do D&C puro). A partir de $\boldsymbol{n \approx 38}$ (em $n = 40$, FB leva 0,1132 ms contra 0,1035 ms do D&C), o crescimento quadrático n^2 supera o *overhead* constante da recursão e a Divisão e Conquista passa a vencer com vantagem crescente. Na Etapa 1, cujo passo de combinação possui menor constante multiplicativa (sem estruturas auxiliares), o cruzamento ocorre ainda mais cedo, em $\boldsymbol{n \approx 6}$ (em $n = 4$, FB vence por 0,0026 ms contra 0,0029 ms; já em $n = 8$, D&C vence por 0,0058 ms contra 0,0065 ms).

Justificativa Teórica e Experimental do Limiar Escolhido ($k = 24$) na Etapa 2: Em uma árvore recursiva binária de tamanho $n = 3.200$, mais de 85% de todas as chamadas recursivas ocorrem nos últimos 4 níveis próximos às folhas (onde os subproblemas têm tamanho ≤ 25). Se pararmos a recursão quando o subproblema atinge um limiar $n \leq k$ e resolvermos essas folhas diretamente por Força Bruta, poda-se uma altura $\Delta h = \log_2(k/3)$ da árvore de recursão, eliminando ~87% das chamadas de função e alocações de listas de partição $P_{y,E}$ / $P_{y,D}$, ao custo de $O(n \cdot k)$ operações leves de distância nas folhas.

A varredura experimental na **Figura 4 (direita)** para $n = 3.200$ avaliou $k \in \{3, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256\}$. O tempo médio cai monotonicamente de **12,963 ms** (em $k = 3$) para um vale ótimo entre $k = 12$ (10,135 ms), $k = 16$ (10,141 ms) e $\boldsymbol{k = 24}$ **(10,071 ms, mínimo global)**, antes de voltar a subir para 11,231 ms ($k = 32$), 15,492 ms ($k = 64$) e 45,171 ms ($k = 256$), quando o termo quadrático k^2 passa a dominar as folhas. Notamos que o limiar ótimo $\boldsymbol{k = 24}$ situa-se ligeiramente abaixo do ponto de cruzamento bruto ($n \approx 38$) porque o algoritmo híbrido já realizou a ordenação inicial (que faz parte do custo fixo de D&C); assim, dentro da árvore recursiva, o ponto em que uma única chamada de Força Bruta supera duas chamadas filhas + partição de P_y + varredura da faixa ocorre exatamente na faixa de 16 a 24 pontos. Essa escolha proporciona uma **redução consistente de 22,4% a 36,6% no tempo total de execução** em relação ao D&C puro em todas as escalas testadas.